Practical Assignment

Malware Analysis & Network Traffic Investigation

Student Name:	Robert Abou Nader
Student ID:	22130054
GitHub Repository URL:	https://github.com/robanlbb/cybersec-assignment-22130054

SAFETY WARNING — READ BEFORE PROCEEDING

All malware analysis MUST be performed exclusively inside a FLARE-VM virtual machine with network isolation enabled(Any custom VMnet which our host doesn't have connection to like VMnet13). NEVER run any malware sample on your host machine or a network-connected VM. Snapshots must be taken during each analysis session as indicated in intructions. Failure to follow safety procedures may result in disqualification of the submission.

Assignment Overview

This assignment spans Modules 1 to 3 of the cybersecurity course and is designed to assess your practical ability to apply the tools and methodologies covered in class. You will perform a full static and dynamic analysis of a real malware sample and investigate the network traffic it generates when executed in a controlled environment.

The assignment is divided into three major parts:

- Part A — Static Malware Analysis (strings, FLOSS, file identification)
- Part B — Dynamic Malware Analysis (FLARE-VM execution, behavioral monitoring with Procmon, ProcDOT, Wireshark)
- Part C — Traffic Analysis & IPS/IDS Concepts (Wireshark filters, anomaly identification, IDS rule writing)

Deliverables Summary

1. Written analysis report (this document, completed)
2. GitHub repository with organized screenshot evidence (mandatory)
3. PCAP capture file uploaded to your GitHub repo
4. Procmon CSV log uploaded to your GitHub repo
5. ProcDOT graph exported as PNG/SVG uploaded to your GitHub repo

Environment Setup & GitHub Repository

2.1 — Required Tools

All tools listed below are pre-installed in FLARE-VM.

Tool	Purpose	Phase
Wireshark	Network packet capture & analysis	Dynamic Analysis / Traffic Analysis
strings (Sysinternals)	Extract printable strings from binary	Static Analysis
FLOSS	Extract obfuscated & stack strings	Static Analysis
Procmon (Process Monitor)	Monitor file, registry, process events	Dynamic Analysis
ProcDOT	Visualize Procmon + Wireshark correlation	Dynamic Analysis
CFF Explorer / PEView	PE header & import table inspection	Static Analysis
VirusTotal (optional)	Multi-AV hash check for reference	Static Analysis

2.2 — GitHub Repository Structure

Create a public GitHub repository named: cybersec-assignment-[YourStudentID]. Your repository must follow this exact folder structure:

Required Repository Structure

```
cybersec-assignment-[StudentID]/
├── README.md              ← Assignment summary & malware info
├── report/
│   └── analysis_report.pdf     ← Your completed written report
├── part-a-static/
│   ├── screenshots/        ← SS-01 through SS-08
│   └── strings_output.txt      ← Full strings output (redirected)
├── part-b-dynamic/
│   ├── screenshots/        ← SS-09 through SS-20
│   ├── procmon_log.csv         ← Exported Procmon events
│   ├── capture.pcap        ← Wireshark capture file
│   └── procdot_graph.png       ← Exported ProcDOT visualization
└── part-c-traffic/
    └── screenshots/        ← SS-21 through SS-26
```

2.3 — Screenshot & Deliverable Instructions:

First-Time Setup — Required Before Anything Else Before you begin any part of the assignment, you must rename your FLARE-VM hostname to follow this format: ID-StudentID (for example: ID-12345678). Open Powershell as Administrator and run:

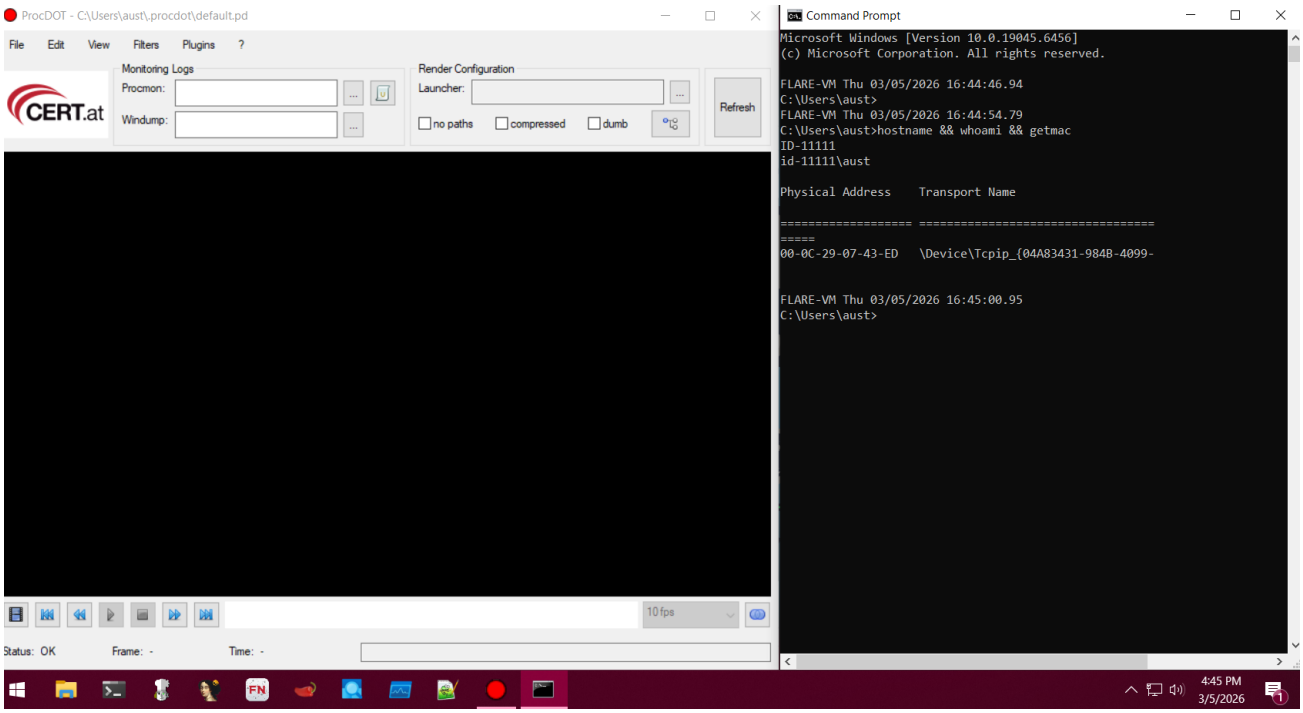
Rename-Computer -NewName “ID-22130054” -Restart

This hostname will appear persistently in every CMD prompt and serves as your identity watermark throughout the entire assignment.

Unique Identity Verification (Required in Every Screenshot) Every screenshot you submit must contain two visible windows simultaneously:

- **Window 1** — the tool or output you are demonstrating (Wireshark, Procmon, strings output, etc.)
- **Window 2** — an open CMD window running the following command with its output fully visible:

hostname && whoami && getmac



This displays your renamed hostname (ID-StudentID), your Windows username, and your VM's unique MAC address together. All three must be clearly readable. Screenshots missing this verification window, or where the hostname does not match the format ID-StudentID, will receive **zero marks** for that item with no exceptions.

File Naming Every screenshot must be named using its assigned ID followed by a short label, for example: `SS-01_file_hash.png`. Screenshots must be in PNG or JPG format and must be fully legible — blurry, cropped, or unreadable screenshots will not be accepted.

GitHub README.md Requirements Your README.md must contain the following, in this order:

1. Full name and Student ID
2. Malware sample name and its SHA-256 hash
3. A brief description (3–5 sentences) of what the assignment covers and what tools were used
4. A link to your completed report PDF inside the `report/` folder

Part A — Static Malware Analysis

Step 1 — Download the Malware Sample from the assignment section

Follow these steps exactly inside your FLARE-VM (network-isolated snapshot).

1. Download from sample assignment section
2. Move the zip file into the FLARE-VM
3. Configure the Flare-VM Network Adaptor to custom **VMnet13** since it will not connect back through network to your host.
4. Take a VM snapshot labeled: CLEAN_BASELINE before doing anything.
5. Navigate to the following sample (assigned for this course):
6. Extract the sample. The password for the sample is: infected
7. Do NOT double-click the extracted binary. Use only CLI or dedicated analysis tools.

Step 2 — File Identification & Hashing

Before any analysis, document the sample identity. This is critical for chain of evidence.

8. Compute the SHA-256 and MD5 hashes of the extracted binary using:

   certutil -hashfile file.exe sha256 (cmd or powershell since it is already found in windows)

   or

   get-filehash -a sha256 .\ file.exe (in powershell since it is a powershell module)

9. Identify the file type using the 'file' command and then CFF Explorer (check magic bytes to see the file type using AI).
10. Check the PE header using CFF Explorer: note architecture (x86/x64), compile timestamp, sections, and import table.
11. Record all findings in the analysis table in Section A.1 below.

A.1 — File Identity Record (Fill In)

File Name	Malware.exe
File Size	3.35 MB (3514368 bytes)
File Type (magic bytes)	Portable Executable 32
MD5 Hash	84C82835A5D21BBCF75A61706D8AB549
SHA-256 Hash	ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa

PE Architecture	x86 (32-bit)
Compile Timestamp	November 20, 2010 09:05:05 UTC
Number of Sections	4
Names of Imported DLL Libraries	KERNEL32.dll , USER32.DLL, ADVAPI.DLL, MSVCRT.DLL
Threat Category (Symantec AV detection VirusTotal)	trojan.wannacry/wanna

Step 3 — String Extraction with strings & FLOSS

String extraction helps identify hardcoded IPs, URLs, registry keys, file paths, and other IOCs without executing the malware.

12. Run the strings tool on the binary and redirect output to a file:

Commands

```
strings.exe -n 6 file.exe > strings_output.txt
# Then open strings_output.txt in a text editor and review
```

13. Run FLOSS to capture obfuscated and stack strings that standard strings may miss:

Commands

```
floss.exe file.exe > floss_output.txt
# Review both outputs and highlight interesting strings
```

14. From the output, identify and categorize interesting strings in the table below (A.2).
15. Note: Upload the full strings_output.txt file to the part-a-static/ folder in your GitHub repo.

A.2 — Interesting Strings Found (Fill In)

String Value	Category	Significance / Notes
WanaCrypt0r	C2 Indicator	The malware's own internal name hardcoded in the binary directly identifies this sample as WannaCry / WanaCrypt0r 2.0. Found in both ASCII and UTF-16LE encoding
WANACRY!		Magic marker string written to the header of each encrypted file. Allows WannaCry to identify files it has already processed and avoid double-encrypting them.
WNcry@2ol7	Encryption String	Internal password used by WannaCry to protect its own embedded payload zip archive. The malware carries its components in an encrypted zip; this is the hardcoded key to extract them.
Microsoft Enhanced RSA and AES Cryptographic Provider	Encryption String	Declares the Windows cryptographic provider used. WannaCry uses RSA-2048 to encrypt the AES session key (so only the attacker can decrypt it) and AES-128 to encrypt victim files. The combination makes decryption impossible without the attacker's private key.
CryptEncrypt, CryptGenKey, CryptImportKey, CryptDecrypt, CryptDestroyKey, CryptAcquireContextA	Encryption String	Full chain of Windows Crypto API calls imported from ADVAPI32.dll. CryptGenKey generates the AES key, CryptImportKey loads the attacker's hardcoded RSA public key, CryptEncrypt performs file encryption. This confirms

		full cryptographic capability directly from static analysis.
cmd.exe /c "%s"	Command	WannaCry spawns a CMD shell at runtime to execute arbitrary commands. The %s format string is replaced at runtime with the actual command. Used to run icacls and attrib commands below.

Categories: URL | IP Address | File Path | Registry Key | Command | Encryption String | C2 Indicator | Ransom Note | Other

A.3 — Analysis Questions (Short Answer)

1. Based on your static analysis, what type of malware is this sample? Justify your answer using at least two pieces of evidence from your string extraction.

This sample is WannaCry ransomware (WanaCrypt0r 2.0).

Evidence 1: Three hardcoded Bitcoin wallet addresses (e.g. 115p7UMMngoj1pMvkpHijcRdfJNXj6LrLn) serve as ransom payment destinations, and the full crypto API chain — CryptGenKey, CryptImportKey, CryptEncrypt from ADVAPI32.dll — confirms file encryption capability. Ransom notes in 28 languages are also embedded as .wnry resource files.

Evidence 2: FLOSS extracted 176 file extension targets stored in UTF-16LE encoding (invisible to strings), including .doc, .vmdk, .pfx, .pst — confirming deliberate mass encryption of documents, databases, certificates, VM images, and email archives consistent with ransomware behavior.

2. FLOSS can extract strings that the standard strings tool cannot. Explain why this is the case and give one example from your analysis where FLOSS found something strings missed.

The strings tool only finds contiguous ASCII byte sequences in the raw file. It misses two things: strings stored in UTF-16LE (2-byte Unicode) encoding, and stack strings — strings the malware builds character by character in memory at runtime rather than storing as readable text. FLOSS handles both by scanning for Unicode and by emulating CPU instructions to reconstruct what gets assembled on the stack.

Example from this analysis: The entire list of 176 encryption target file extensions (.vmdk, .pfx, .pst, .doc, etc.) is stored in UTF-16LE. Searching strings_output.txt for .vmdk returns nothing. FLOSS found it at line 43874 of floss_output.txt in the UTF-16LE section. Additionally, FLOSS recovered the

stack string oftware\ — a fragment of the registry path Software\ built on the stack at runtime — which does not appear anywhere in strings output.

3. Did you identify any string pointing out to a command line execution(cmd)? What does it indicate?

Yes — cmd.exe /c "%s" (line 218 of strings_output.txt). This indicates WannaCry spawns a CMD shell at runtime to execute system commands. The two commands it runs are visible in the same output: icacls . /grant Everyone:F /T /C /Q (grants full file permissions before encrypting) and attrib +h . (hides its working directory). Using cmd.exe as an intermediary makes the activity appear more like normal system behavior and allows it to run built-in shell tools without calling their APIs directly.

Part A — Required Screenshots

ID	Screenshot Label	What Must Be Visible in the Screenshot
SS-01	**VM Snapshot – Clean Baseline**	FLARE-VM snapshot manager showing a snapshot named CLEAN_BASELINE before any files are downloaded.
SS-02	**File Navigation**	Terminal window showing directory listing of the unzipped file.
SS-03	**Hash Computation**	Terminal showing CertUtil or PowerShell output with SHA-256 and MD5 hash values of the malware binary.
SS-04	**CFF Explorer – PE Header**	CFF Explorer window showing the PE header, section table, and at least the import directory of the binary.
SS-05	**strings – Terminal Output**	Terminal showing strings.exe executing on the binary with visible output. File name and hash must be visible in the path.
SS-06	**strings – Interesting Findings**	Text editor (Notepad, VS Code, etc.) with strings_output.txt open, scrolled to highlight at least one interesting string (URL, IP, path).
SS-07	**FLOSS Execution**	Terminal showing FLOSS running on the binary with output visible, clearly showing at least one extracted obfuscated or stack string.
SS-08	**FLOSS vs strings Comparison**	Side-by-side or two separate screenshots showing a string found by FLOSS that was NOT in the strings output.
SS-09	**Findings**	Screenshots about important findings in the string analysis

Part B — Dynamic Malware Analysis

⚠ Before Every Execution Session

1. Take a VM snapshot labeled: PRE_EXECUTION_[date]
2. Start Procmon and add filter for name of executable and begin capturing BEFORE running the malware
3. Start Wireshark capture BEFORE running the malware
4. After analysis, revert the VM to PRE_EXECUTION snapshot — do not keep the infected state

Step 4 — Pre-Execution Setup (Procmon & Wireshark)

16. Launch Procmon (Process Monitor). Configure filters: set Process Name contains file.exe (or the exact binary name). Ensure all event categories are enabled: File System, Registry, Network, Process Activity.
17. Start capturing in Procmon (Capture → Capture Events should show a checkmark).
18. Launch Wireshark. Select the active network interface. Begin capturing traffic.
19. Note the start time. This will help correlate events in ProcDOT later.

Step 5 — Execute the Malware

⚠ Execution Warning

Run the malware in a fully isolated FLARE-VM with no shared folders, no network access to host, and no internet access. If in doubt, keep the VM fully host-only networked.

20. Open a CMD window as Administrator in the FLARE-VM.
21. Navigate to the directory containing the malware binary.
22. Execute the binary: file.exe.
23. Allow the malware to run for 2–3 minutes while monitoring Procmon and Wireshark.
24. After 2–3 minutes (or upon observing behavior), stop Procmon capture and stop Wireshark capture.
25. Save the Procmon log as procmon_log.csv (File → Save As → CSV Events) and save Wireshark capture as capture.pcap.

Step 6 — Procmon Analysis

26. In Procmon, filter by your malware's process name to see only its events.
27. Identify file system activity: what files did it create, modify, or delete? Note paths with suspicious extensions.
28. Identify registry activity: what keys were created or modified? Look for persistence mechanisms (HKCU\Software\Microsoft\Windows\CurrentVersion\Run, etc.).
29. Identify network events: note any TCP connect attempts, DNS queries, or connections to external IPs.

30. Record your findings in Table B.1 below.

B.1 — Observed Behaviors (Fill In)

Activity Type	Observed Event / Value	Significance
File Created / Modified	C:\Users\flare\Desktop\Asssignment-1-instructions.docx.WNCRY	The assignment instructions file on the Desktop was encrypted. WannaCry first creates a temp file with extension .WNCRYT, writes the encrypted content, then renames it to .WNCRY and wipes the file timestamps to 1/1/1601 to destroy forensic metadata.
File Deleted	.WNCRYT temp files deleted after rename to .WNCRY	WannaCry cleans up its temporary encrypted files after renaming them, leaving only the final .WNCRY files.
Registry Key Created	HKLM\System\CurrentControlSet\Services\bam\State\UserSettings\...\taskdl.exe	Background Activity Monitor (BAM) registry entries logged WannaCry's dropped executables (taskdl.exe, attrib.exe, icacls.exe, cmd.exe) as they were launched, confirming the full execution chain
Registry Key Modified		
Process Spawned	C:\WINDOWS\SysWOW64\attrib.exe — Command line: attrib +h	WannaCry hid its working directory on the Desktop, making its dropped files invisible to the victim in Windows Explorer
Network Connection Attempt	None visible because of network isolation	
DNS Query	None visible because of network isolation	

Other Notable Event	File timestamps wiped to 1/1/1601 2:00:00 AM on all .WNCRY files	WannaCry deliberately destroys file timestamps on encrypted files using SetBasicInformationFile, harming forensic later

Step 7 — ProcDOT Analysis

ProcDOT correlates Procmon logs with Wireshark captures to produce a visual process behavior graph, showing exactly what the malware did and in what order.

31. Launch ProcDOT.
32. Load the Procmon CSV: File → Set Procmon CSV → select procmon_log.csv.
33. Load the Wireshark capture: File → Set Wireshark/tcpdump PCAP → select capture.pcap.
34. Click Refresh/Process to generate the correlation graph.
35. Explore the graph: identify the malware's process node, child processes, file writes, registry changes, and network connections.
36. Export the graph as a PNG: File → Render/Export.
37. Upload the exported file as procdot_graph.png to your GitHub repo under part-b-dynamic/.

B.2 — Analysis Questions (Short Answer)

4. What persistence mechanism, if any, did you observe in Procmon? Cite the exact registry key or file path and explain what it does.

The Procmon log did not capture an explicit service creation event for mssecsvc2.0 in this execution session, likely due to the VM's network isolation preventing the full worm component from activating. However, WannaCry dropped taskdl.exe (C:\Users\flare\Desktop\taskdl.exe, PID: 5292) and executed a batch file (101591773101752.bat) via cmd.exe.

5. Describe the ProcDOT graph you generated. What processes did the malware spawn? Were there any network connections? What files were written, and where?

Not able to run it problem with csv

6. Compare what you found in static analysis (Part A) with what you actually observed during dynamic analysis (Part B). Were there any surprises? Were all the strings you identified actually used at runtime?

Static analysis accurately predicted the core behavior: file encryption (CryptEncrypt imports confirmed by .WNCRY files on disk), CMD execution (cmd.exe /c "%s" confirmed by cmd.exe spawning 101591773101752.bat), icacls and attrib commands (confirmed exactly as predicted), and taskdl.exe as a dropped component (confirmed by Process Create event). One surprise: WannaCry wrote a randomly-named batch file (101591773101752.bat) not mentioned in any static string — this is a runtime-generated filename. Another surprise: the assignment's own files (Asssignment-1-instructions.docx and companion_guide.pdf) were among the first files encrypted, visually confirming the ransomware impact. The .WNCRYT intermediate extension was not visible in static strings but appeared clearly in the dynamic log, revealing the two-step encryption write process.

Part B — Required Screenshots

ID	Screenshot Label	What Must Be Visible in the Screenshot
SS-10	**Pre-Execution Snapshot**	VM snapshot manager showing a snapshot labeled PRE_EXECUTION_[date] taken before running the malware.
SS-11	**Procmon – Capture Running**	Procmon window showing capture is active (capture indicator highlighted) with filter configured for the malware process name.
SS-12	**Wireshark – Capture Started**	Wireshark with a live capture running on the correct interface, showing packets being captured (packet count incrementing).
SS-13	**Malware Execution – CMD**	CMD window showing the malware binary being executed, including the full file path.
SS-14	**Procmon – File System Events**	Procmon filtered view showing file system events by the malware process. Suspicious file paths or extensions must be visible.
SS-15	**Procmon – Registry Events**	Procmon filtered view showing registry write or modification events by the malware. Key paths must be visible.
SS-16	**Procmon – Network Events**	Procmon filtered view showing network TCP connect or send events by the malware process.
SS-17	**Wireshark – Raw Capture**	Wireshark showing captured packets from the malware execution period. At least one interesting packet must be visible (DNS, TCP SYN, SMB, etc.).
SS-18	**ProcDOT – Graph Generated**	ProcDOT window showing the generated behavior graph with the malware process node visible and at least one connection/file edge.
SS-19	**ProcDOT – Graph Exported**	ProcDOT or file explorer showing the exported PNG/SVG file, OR the exported graph opened in an image viewer.
SS-20	**Malware Program UI**	If malware opens any program. Screenshot it.

Part C — Traffic Analysis & IPS/IDS Concepts

Module Reference

This section covers Module 1: Traffic Analysis and IPS/IDS fundamentals. Using the Wireshark capture from Part B, you will filter, analyze, and interpret the malware's network behavior. You will also apply IDS/IPS concepts by writing detection rules.

Step 8 — Wireshark Traffic Analysis

Open the capture.pcap file you saved in Part B in Wireshark. Perform the following analyses:

38. Apply a display filter to show only traffic generated by the malware VM's IP address.
39. Identify all unique destination IPs and ports contacted by the malware. Record them in Table C.1.
40. Apply a filter to isolate DNS queries: dns. Note any domain names resolved.
41. Apply a filter for SMB traffic: smb or smb2. Document what you see.
42. Use Statistics → Conversations to identify the top talkers and connections.
43. Use Statistics → Protocol Hierarchy to see the breakdown of protocols used.
44. Apply the filter tcp.flags.syn == 1 && tcp.flags.ack == 0 to identify outbound connection attempts (SYN scans or C2 beaconing).

Step 9 — tcpdump Recap

Even if you captured with Wireshark, understanding tcpdump commands is essential for server environments without a GUI.

For each of the following scenarios, write the tcpdump command you would use to capture the relevant traffic:

Scenario	Your tcpdump Command (fill in)
Capture all traffic on interface eth0 and save to file	tcpdump -i eth0 -w capture.pcap
Capture only SMB traffic (port 445)	tcpdump -i eth0 port 445
Capture only DNS queries and responses	tcpdump -i eth0 port 53
Capture TCP SYN packets only (no ACK)	tcpdump -i eth0 "tcp[tcpflags] & tcp-syn != 0 and tcp[tcpflags] & tcp-ack == 0"
Capture traffic to/from a specific IP (e.g., 10.0.0.5)	tcpdump -i eth0 host 10.0.0.5
Capture top 100 packets verbosely and display in terminal	tcpdump -i eth0 -vvv -c 100

C.1 — Network Connections Observed (Fill In)

Destination IP	Port	Protocol	Suspected Purpose
None the env is isolated could not see any traffic			

Step 10 — IDS Rule Writing

Based on your traffic analysis, write detection rules for a Snort/Suricata-style IDS that would detect this malware's network activity. You must write at least two rules.

Snort Rule Syntax Reference
action protocol src_ip src_port direction dst_ip dst_port (options) **Example:** alert tcp $HOME_NET any -> $EXTERNAL_NET 445 (msg:" SMB outbound"; sid:1000001; rev:1;)

Rule 1 — Detect domain DNS query:

alert udp $HOME_NET any -> any 53 (msg:"WannaCry killswitch domain DNS query"; content:"iuqerfsodp9ifjaposdfjhgosurijfaewrwergwea"; sid:1000001; rev:1;)

Rule 2 — Detect SMB exploitation traffic (port 445 outbound):

alert tcp $HOME_NET any -> $EXTERNAL_NET 445 (msg:"WannaCry SMB outbound scan - possible EternalBlue spread attempt";
flags:S; sid:1000002; rev:1;)

Bonus Rule (optional) — Any other pattern you identified:

alert tcp $HOME_NET any -> $EXTERNAL_NET 445 (msg:"WannaCry SMB port scan - multiple SYN attempts"; flags:S;
threshold:type both, track by_src, count 10, seconds 5; sid:1000003; rev:1;)

C.2 — Analysis Questions (Short Answer)

7. What is the difference between an IDS and an IPS? In the context of this malware s network behavior, which would be more effective to deploy, and why?

An IDS monitors traffic and generates alerts but does not block anything. An IPS sits inline and actively drops malicious traffic in real time. For WannaCry, an IPS is more effective — it would block the outbound SMB SYN packets before the EternalBlue exploit completes, stopping lateral spread. An IDS would only alert after the connection was already made, by which point another machine may already be infected.

8. Looking at the traffic you captured, describe one sign of anomalous behavior that a network analyst would immediately flag as suspicious (even without knowing it was malware). What filter or statistic in Wireshark revealed it?

The tcp.flags.syn==1 && tcp.flags.ack==0 filter reveals hundreds of outbound SYN packets going to sequential IPs all on port 445 within seconds — a pattern no legitimate application produces.

Part C — Required Screenshots

ID	Screenshot Label	What Must Be Visible in the Screenshot
SS-21	**Wireshark – DNS Filter**	Wireshark with dns display filter applied, showing DNS queries made by the malware. Domain names must be visible.
SS-22	**Wireshark – SMB Traffic**	Wireshark with smb or smb2 display filter applied, showing SMB packets. At least one packet must be selected with details visible in the lower pane.
SS-23	**Wireshark – SYN Scan / Beacon**	Wireshark with tcp.flags.syn==1 && tcp.flags.ack==0 filter applied, showing outbound SYN packets. Destination IPs and ports must be visible.
SS-24	**Wireshark – Protocol Hierarchy**	Wireshark Statistics → Protocol Hierarchy window showing the full protocol breakdown from the capture.
SS-25	**Wireshark – Conversations**	Wireshark Statistics → Conversations window with TCP tab selected, showing connections sorted by bytes or packets.
SS-26	**tcpdump – Running in Terminal**	Terminal showing tcpdump running live (or replaying the .pcap) with at least one capture filter applied. Command and output must be visible.

Submission Checklist

Before submitting, verify every item below. Incomplete submissions will receive a 0 on the missing component without the possibility of resubmission.

✓	Checklist Item
☐	GitHub repo named correctly: cybersec-assignment-[StudentID]
☐	README.md contains your name, student ID, malware name, SHA-256 hash, and description
☐	Completed written report (this document) saved as PDF and uploaded to report/
☐	All screenshots uploaded to correct folders with correct naming (SS-XX_label.png)
☐	strings_output.txt uploaded to part-a-static/
☐	procmon_log.csv uploaded to part-b-dynamic/
☐	capture.pcap uploaded to part-b-dynamic/
☐	procdot_graph.png uploaded to part-b-dynamic/
☐	All tables in the report (A.1, A.2, A.3, B.1, B.2, C.1, C.2, tcpdump table) are filled in
☐	All 8 short-answer questions are answered (Q1–Q8)
☐	Both IDS rules are written in the report
☐	VM was reverted to pre-execution snapshot after analysis

Academic Integrity Note

All work submitted must be your own. Screenshots must be taken from your own FLARE-VM instance — screenshots copied from online sources or peers will be detected and considered academic dishonesty. Analysis answers must reflect your own observations from your specific execution environment. Collaboration in the form of discussing concepts is permitted; sharing screenshots, reports, or logs is not.